



Tribhuvan University
Faculty of Humanities and Social Sciences

A PROJECT REPORT FOR
“ByteStore
An Intelligent E-Commerce Recommendation System”

Submitted to
Department of Computer Application

Advanced College of Engineering and Management
Kalaki, Kathmandu

In the partial fulfillment of the requirements for
Bachelors in Computer Application

Submitted by:
Abhinav Sapkota (002)

10/4/2026

Table of Contents

1. Introduction.....	1
2. Problem Statement	2
3. Objectives	2
4. Methodology	3
4.1. Requirement Identification	3
4.1.1. Study of Existing System.....	3
4.1.2. Literature Review.....	4
4.1.3. Requirement Analysis	6
4.2. Feasibility Analysis.....	7
4.2.1. Technical Feasibility:	7
4.2.2. Economic Feasibility:	8
4.2.3. Operational Feasibility:.....	8
4.3. High Level Design of the Proposed System	10
4.3.1. Methodology of the Proposed System.....	10
4.3.2. Working Mechanism of the Proposed System	11
4.3.3. Flow Chart	12
4.3.4. System Design	14
4.3.5. Description of Algorithms	15
5. Gantt Chart	18
6. Expected outcome	19
7. References.....	20

List of Figures

Figure 4.2.1:Use Case Diagram of the ByteStore Recommendation Module	9
Figure 4.3.1:Evolutionary / Incremental Development model for ByteStore.....	11
Figure 4.3.2:System Diagram for ByteStore Recommendation System.....	12
Figure 4.3.3:Flow chart for Bytestore.....	13
Figure 4.3.4Architectural design for Bytestore.....	14
Figure 5.1: Gantt Chart for Bytestore	18

List of Tables

Table 5.1:Scheduling Table for Bytestore.....	18
---	----

1. Introduction

Buying things online has become part of everyday life for a lot of people, and Nepal is no exception. More households now shop for phones, laptops, and accessories from home rather than visiting a shop in person, and that shift has only picked up pace over the last few years. ByteStore is a web-based e-commerce platform built to serve exactly this kind of shopping a place where customers can browse technology products, read specifications, add items to a cart, and check out, while store staff manage stock and orders from an admin panel behind the scenes. It is built using PHP, MySQL, HTML, CSS, and JavaScript, a combination that is common for this kind of system because it is well documented, free to use, and capable of handling everything from user accounts to secure checkout.

Most online stores, ByteStore included in its basic form, work the same way for every visitor. A customer types in what they want, filters by category if they know exactly what they're looking for, and scrolls through whatever comes back. This works fine when a catalog is small, but it starts to break down as more products get added, since customers have to do all the filtering themselves with no help from the platform. Two customers with completely different buying habits will still see the same homepage, the same order of products, and the same suggestions, which in this case means no suggestions at all.

This project sets out to change that part of ByteStore specifically. Instead of leaving product discovery entirely up to the customer, the platform is being extended with a recommendation system that looks at what a customer has already bought, and what kind of products they tend to go for, to suggest things they are more likely to want. This is done using two techniques working together, Collaborative Filtering and Content-Based Filtering, both relying on a similarity measurement method called Cosine Similarity, which is explained in full later in this proposal. The goal is not to reinvent how e-commerce works, but to bring ByteStore a step closer to how larger, more established platforms already operate.

2. Problem Statement

Most online shopping today still comes down to a customer searching, filtering by category, and scrolling through a long list of products until they find something they like. As online catalogs keep growing larger, this becomes harder to do, and customers end up spending more time browsing than actually deciding what to buy. This problem is not specific to any one store, it is a general issue with online shopping platforms have access to a lot of information about what customers do, but a lot of that information is never actually used to help the customer.

This is not just an inconvenience for the customer, it is also a missed opportunity for the store. Every extra minute spent searching is a minute where a customer might simply give up and leave without buying anything, especially if a competing platform makes it easier to find what they want. On the store's side, the purchase and browsing data needed to fix this is often already being collected, it is just never used again once the order is placed. This pattern shows up across many existing systems, not only ByteStore, from basic student-built e-commerce projects to larger platforms that only partially personalize what customers see. The result is the same industry-wide habit repeating itself: systems keep the data, but rarely act on it.

This is a real gap. A customer who has bought three gaming laptops in a row is, statistically speaking, probably going to be interested in a gaming mouse or a laptop cooling pad, but a plain e-commerce system with no personalization will show that customer the exact same homepage as someone who has never bought electronics before. The result is a slower, less useful shopping experience than it could be, and stores lose a chance to point customers toward things they would likely buy.

3. Objectives

This project has one general objective and one specific objective:

1. To build ByteStore as a complete, working e-commerce platform that gives customers an easy way to browse and buy tech products, and gives admins and employees the tools to manage products, orders, and users.
2. To add a product recommendation system to this platform, using Collaborative Filtering and Content-Based Filtering, both calculated with Cosine Similarity, so customers get personalized suggestions instead of the same listing shown to everyone.

4. Methodology

This project follows the Evolutionary / Incremental Development model. Under this model, a basic working version is built first, and then improved in stages, with room to revisit and adjust earlier parts as needed. This fits a solo project well, since it does not require locking down every requirement upfront the way a strict Waterfall approach would.

4.1. Requirement Identification

4.1.1. Study of Existing System

Before deciding on an approach, it helps to look at how other e-commerce systems are built, both ones that personalize nothing and ones that already do.

One example is a plain PHP and MySQL e-commerce project on GitHub, built without any external framework, offering registration, a cart, order management, and an admin panel. It covers the basic CRUD operations expected of an online store, but every customer sees the same product listing regardless of what they've bought before. A second example is another full-stack PHP platform, also on GitHub, with a more advanced admin dashboard and AJAX-based cart handling, but it still has no way of tailoring what a customer sees. On the other end, there are real e-commerce systems already using the same kind of algorithms this project is built around. Etsy, for example, is known to combine content-based filtering, looking at a listing's own attributes like category and style, with collaborative signals based on what similar shoppers have viewed or bought, to decide what to show a visitor on its homepage and search results. This shows that the two techniques used in this project are not just theoretical, they are already running in production on a large online marketplace. A smaller-scale example is an open student and independent project on GitHub that builds a PHP and MySQL e-commerce prototype paired with collaborative filtering, content-based filtering, and a hybrid of both, which is very close in spirit to what ByteStore is doing here, just smaller.

ByteStore, the platform this project is built around, currently works like the first two examples. It is a fully working PHP and MySQL store with the same core functionality, but no personalization yet. This project adds that missing piece, moving ByteStore from the first kind of system toward the second.

ByteStore currently works like the first two examples: a fully working PHP and MySQL store with no personalization yet. This project moves it toward the second kind, adding that missing piece using data the platform already collects but doesn't yet use.

4.1.2. Literature Review

A fair amount of research and real-world work already exists around recommendation systems in e-commerce, and reviewing it helps justify why Collaborative Filtering and Content-Based Filtering were chosen for this project instead of just relying on plain search and filters.

"PLING Adaptive Opportunity Checking of Web Service with Recommendation System" published in the International Journal of Pure and Applied Mathematics, this paper presents a hybrid filtering approach combining collaborative and content-based filtering, using cosine similarity to analyze user browsing history and content similarity. It argues that combining both methods handles the growing complexity of information filtering better than using just one. This closely matches the direction of this project. [1]

The Amazon.com recommendations paper describes how item-to-item collaborative filtering, using cosine similarity, is used by Amazon to power its "customers who bought this also bought" feature, without requiring any personal information from the customer beyond their purchase history. It is one of the clearer real-world explanations of why this technique is popular for online stores specifically. [2]

"A Hybrid Collaborative Filtering Method for Multiple-Interests and Multiple-Content Recommendation in E-Commerce" published in Expert Systems with Applications by Yu Li, Liu Lu, and Li Xuefeng, this is an older but still frequently cited paper describing how combining collaborative filtering with content information improves recommendation quality in online shopping settings. [3]"A Personalized Collaborative Recommendation Algorithm Based on Hybrid Information" presented at the International Conference on Mechatronics, Materials, Chemistry and Computer Engineering by Dejie Zhang, this paper points out a real weakness in plain collaborative filtering as the number of customers and products grows, finding the "nearest neighbor" customers becomes slow and less reliable. It proposes combining user and item information together to keep recommendations fast and accurate even as a store's catalog grows, which is a relevant consideration even at ByteStore's smaller scale. A more general survey on hybrid recommender systems lays out several different ways collaborative and content-based methods can be combined, and shows that hybrid approaches consistently perform better than either method on its own across different kinds of systems, not just e-commerce. This is one of the most widely cited papers on the topic and supports the direction of combining both methods in this project as well. [5]

Hybrid recommendation study with Amazon Electronics and MovieLens datasets this more recent paper builds a hybrid model combining collaborative filtering, matrix factorization, and a content-based fallback for new customers, tested on real Amazon Electronics purchase data. It notes that recommended products already make up a significant share of Amazon's revenue, which is a useful, concrete number showing why stores keep investing in this feature instead of treating it as optional polish. [6]

Brazilian E-Commerce Public Dataset by Olist a widely used dataset containing real sales data from about 100,000 orders, including customer behavior and order items. It has been used in many public notebooks to study buying patterns and build recommendation models, and is a good reference point for how order and product data, similar to what ByteStore already stores, can be turned into recommendations. [7]

E-Commerce Purchase History Dataset, Electronics Store this dataset is particularly relevant since it focuses specifically on an electronics store, which lines up closely with ByteStore's own catalog. It has been used in student and independent projects to build and compare collaborative filtering and content-based filtering models on real purchase behavior. [8]"Building and Optimizing a Personalized Recommendation System for E-Commerce Using Machine Learning" this project is worth mentioning because it is close in spirit to this one: a student-built e-commerce prototype using PHP, MySQL, and XAMPP, paired with collaborative filtering, content-based filtering, and a hybrid model. Its results show that collaborative filtering performed noticeably better than content-based filtering alone in terms of accuracy, but content-based filtering still held up better for newer users, which supports the decision to use both together rather than picking one. [9]

"A Content-Based Recommender for E-Commerce Web Store" a published data science project write-up that builds a content-based recommender for an online store using product category and price information. It lays out the trade-off clearly: content-based filtering avoids the cold-start problem but depends entirely on having good product attribute data, while collaborative filtering can find surprising, less obvious recommendations but needs a decent amount of purchase history to work well. This trade-off is the main reason this project is not relying on just one of the two methods. [10]

Taken together, these sources support the same idea from a few different directions: relying on plain search and category browsing leaves a lot of useful customer data unused, collaborative filtering and content-based filtering solve different halves of the same problem, and using both together, tied by cosine similarity, is a well-established and realistic approach rather than something overengineered for a project of this scope.

4.1.3. Requirement Analysis

Functional Requirements:

Admin

1. Can add, edit, delete and view all users
2. Can add, edit, delete and view all employees
3. Can add, edit, delete and view all products
4. Can login
5. Can view details of orders
6. Can see which products are being recommended most often, through the dashboard

Customer

1. Can login or register
2. Can buy products
3. Can search for products
4. Can add products to cart

Employee

1. Can add, edit, delete and view all products
2. Can view all users
3. Can login
4. Can view details of orders

System

1. The system should build a record of which products each customer has purchased.
2. The system should build a profile for each product based on attributes such as category, brand, and price range.
3. The system should calculate similarity scores using cosine similarity, comparing customers to each other for collaborative filtering, and comparing products to each other for content-based filtering.
4. The system should generate a combined, ranked list of recommended products for each customer.

Non-Functional Requirements:**Security**

This system has accounts for its users and only authorized users can access the system with a username and password. Passwords are encrypted using PHP's PASSWORD_DEFAULT function.

Availability

This system is available to users anytime, anywhere, just need a PC or Mobile and Internet Connection. Also, the system works in multiple web browsers like (Chrome, Mozilla and Opera).

Maintainability

The system will be easily maintained by the developer or other authorized trained person and Backup for database can be done easily.

Recommendation-System

Recommendations should load quickly enough that they do not noticeably slow down the site. The recommendation module should be built using Object-Oriented Programming, so it stays organized and easy to extend later.

Resource Requirements

Unlike a full machine learning system, this recommendation module does not need special hardware, a GPU, or a long training process, since Collaborative Filtering and Content-Based Filtering here are similarity calculations run directly on existing data rather than a model trained over time. The only real resource cost is a small amount of extra processing on each page load, since the system needs to calculate similarity scores before showing recommendations. For ByteStore's catalog and customer base size, this is expected to add at most a few hundred milliseconds to a page's load time, not a noticeable delay for the end user.

4.2. Feasibility Analysis

Feasibility study checks whether the project is practical, cost-effective, and achievable.

4.2.1. Technical Feasibility:

The system uses PHP, MySQL, HTML, CSS, and JavaScript, which are freely available and supported by XAMPP. Tools are easy to learn, making implementation technically possible.

4.2.2. Economic Feasibility:

Since it is an academic project, costs are minimal (local hosting, no domain/hosting required yet). Development requires only time and effort, so it is economically viable.

4.2.3. Operational Feasibility:

The system will be simple for customers to browse and order products, and for employees to manage product listings. Recommendations are shown as a simple list of products on pages a customer already visits, so there is nothing new to learn for someone already using the site. It meets both user needs and project objectives, making it operationally feasible.

Scope and Limitations

The scope of this project is limited to adding a recommendation layer on top of the existing ByteStore platform. This includes building customer purchase profiles, product attribute profiles, calculating similarity scores using Cosine Similarity, and combining Collaborative Filtering and Content-Based Filtering into one ranked recommendation list shown to the customer, along with a basic admin view of recommendation activity.

This project does not attempt to rebuild ByteStore's existing core features, such as authentication, cart, checkout, or payment processing, since these already exist and work. It also does not include more advanced recommendation techniques such as matrix factorization or deep learning based models, which are better suited to much larger catalogs and customer bases than ByteStore currently has. Other extensions considered early in the planning of this project, such as delivery route optimization and demand forecasting, were left out to keep the project focused and achievable within the time and credit weight of Project II.

The recommendation system is also limited by the data available to it. Since ByteStore is a student project rather than a live commercial store, it does not have the scale of real purchase data that platforms like Amazon or Etsy use to refine their systems. Seed data will be used to demonstrate and test the recommendation logic, which may not fully reflect how the system would behave with a much larger, real-world customer base.

Use case diagram

The system involves three actors: Customer, Employee and Admin. The customer sees recommended products while browsing or after checkout, while the admin can view recommendation activity through the dashboard.



Figure 4.2.1: Use Case Diagram of the ByteStore Recommendation Module

4.3. High Level Design of the Proposed System

4.3.1. Methodology of the Proposed System

The Evolutionary / Incremental model is followed here: the recommendation logic is built first, then the customer-facing display, and finally the admin-facing reporting view, with each stage tested before moving to the next.

The Evolutionary / Incremental Development model for software development

This is the model that will be used to develop ByteStore. Feedback loops are allowed throughout the whole development process. Since this is a solo project, a model that does not require a large team fits well, and this is why the Evolutionary / Incremental model was chosen.

Under this model, software is built and refined through several versions rather than all at once. The process generally follows these stages:

1. The problem is analyzed and requirements are identified.
2. The specifications of the project are set once the requirements are clear.
3. A design for the software system is created.
4. Implementation and debugging begin once the design is complete.
5. The program is tested and the system is integrated.
6. The finished software moves into its final stage, with operation and maintenance continuing afterward.

Because this project is being developed by a single person, this model is especially practical: each stage can be revisited without needing to coordinate with a team, and anything learned while testing the recommendation module can be folded back into the design right away, rather than waiting for a scheduled review point. This keeps mistakes small and easy to catch early, instead of only surfacing at the very end of the project.

This process is shown below as a top-down diagram. Feedback loops are added between stages so that the whole process stays coordinated as the project moves forward.

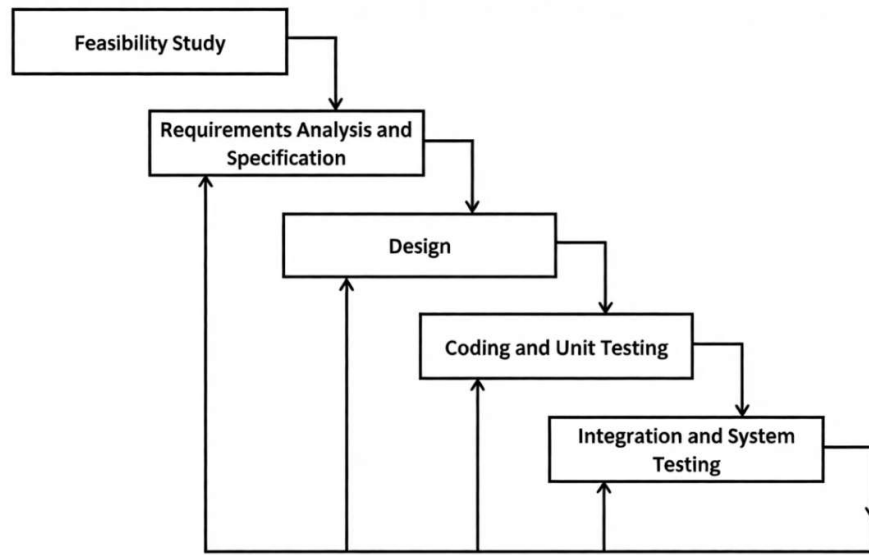


Figure 4.3.1: Evolutionary / Incremental Development model for ByteStore

4.3.2. Working Mechanism of the Proposed System

The working mechanism of ByteStore describes how its major components interact once the recommendation module is added. Customers, employees, and admins all interact with the system through the Backend, which handles requests such as browsing, placing orders, managing products, and managing users. The Backend stores and retrieves data from the Database as it always has. What's new is a separate Recommendation Engine that the Backend talks to: whenever recommendations need to be generated, the Backend sends the relevant purchase and product data to the Recommendation Engine, which runs Collaborative Filtering and Content-Based Filtering, both using Cosine Similarity, and sends back a ranked list of recommended products. This keeps the recommendation logic as a distinct, self-contained part of the system rather than mixed into the rest of ByteStore's existing code.

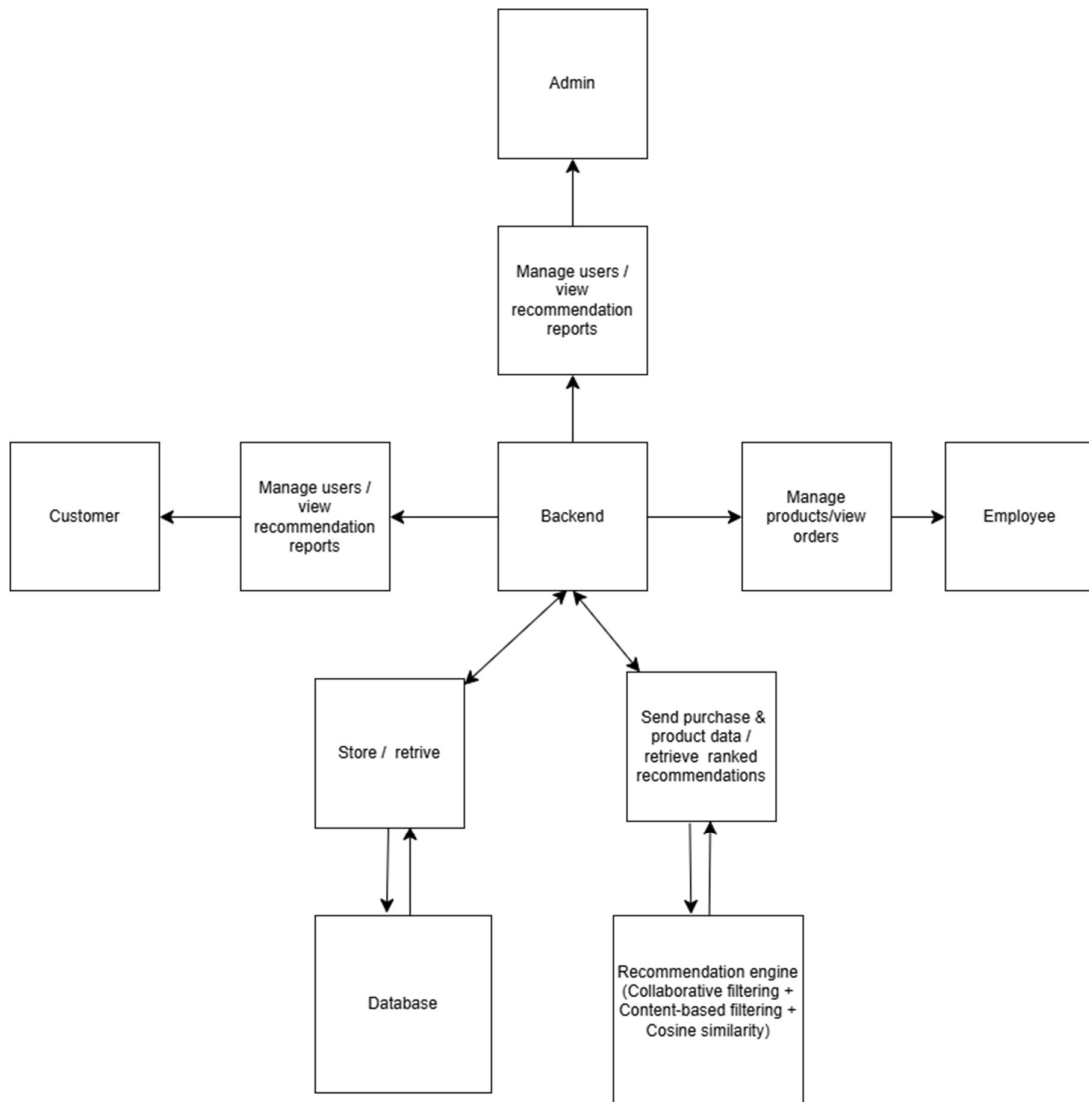


Figure 4.3.2: System Diagram for ByteStore Recommendation System

4.3.3. Flow Chart

The flow below shows what happens when a customer visits the site: the system checks their purchase history for collaborative filtering, checks product attributes for content-based filtering, calculates similarity scores for both, and combines the results into one ranked list of recommended products.

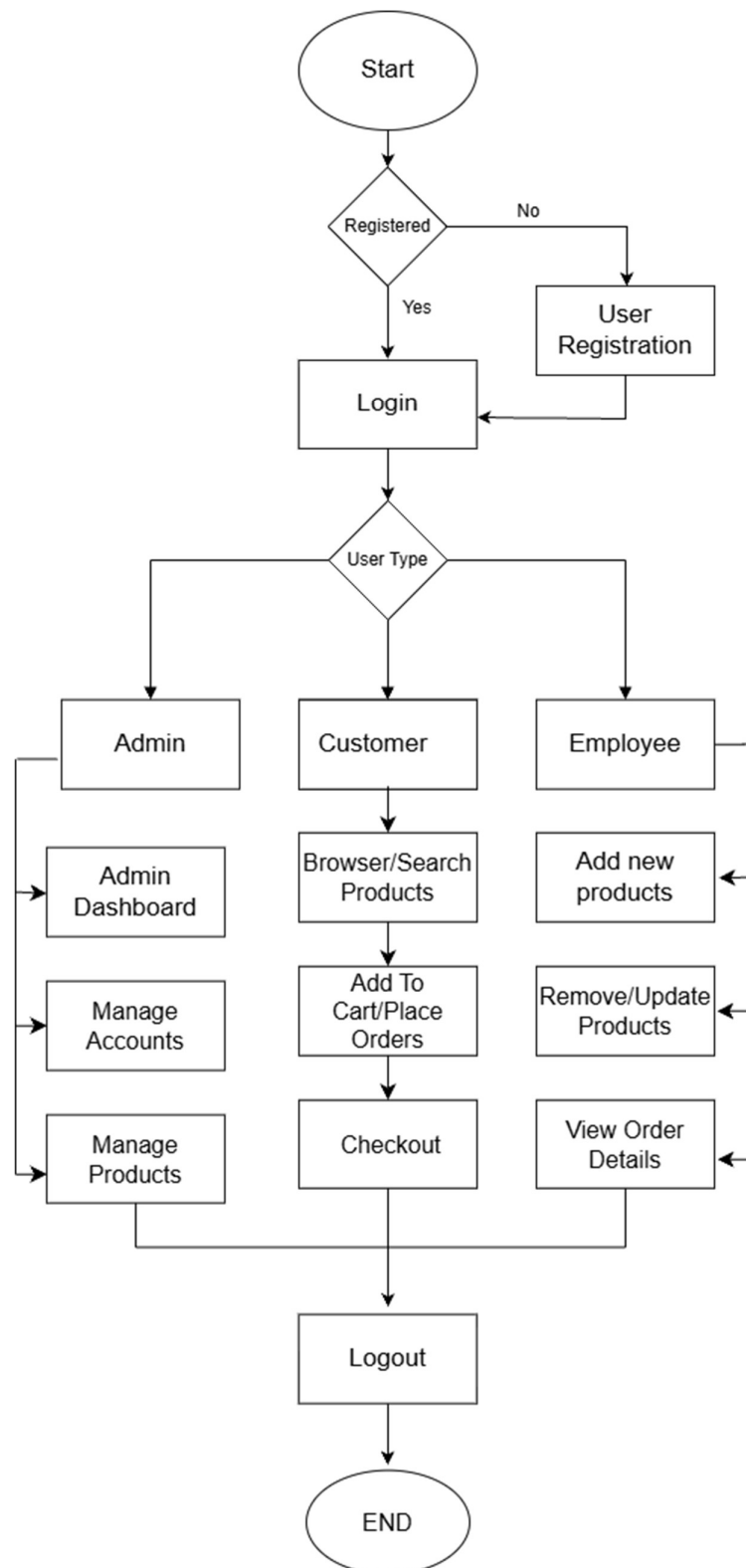


Figure 4.3.3:Flow chart for Bytestore

4.3.4. System Design

Architectural Design

ByteStore follows a layered architecture, separating the user interface from the business logic and from the data itself. The presentation layer (HTML, CSS, and JavaScript) handles what the customer and admin actually see and interact with. The business logic layer, written in PHP, handles everything in between: processing orders, managing products, and now also running the recommendation module's Collaborative Filtering, Content-Based Filtering, and Cosine Similarity calculations. The data layer is a MySQL database, which the business logic layer queries and updates as needed. Keeping these layers separate means the recommendation module can be added, changed, or extended without needing to touch the presentation layer or the database structure underneath it.

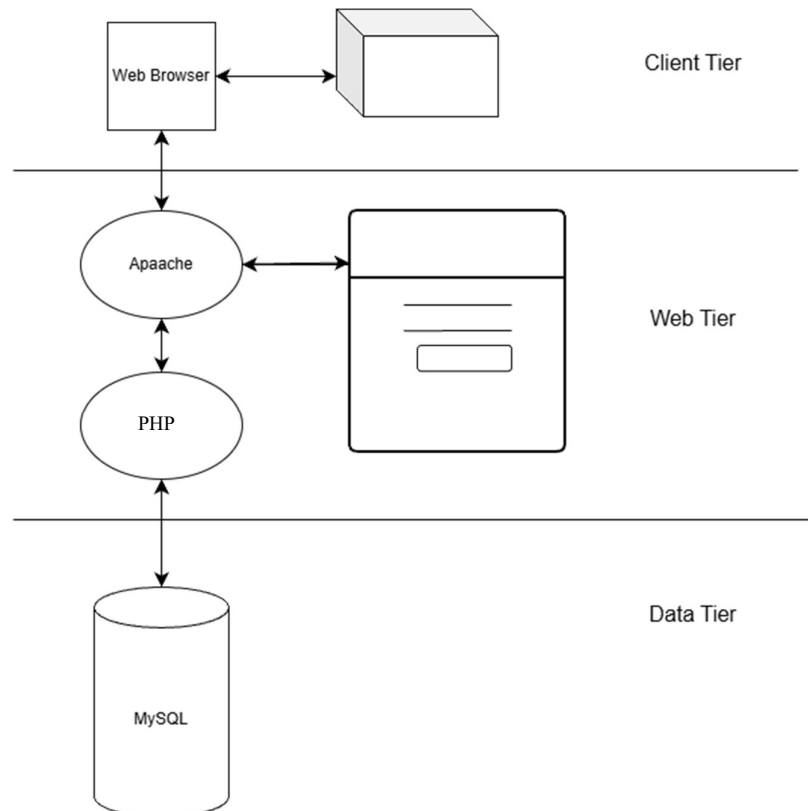


Figure 4.3.4. Architectural design for Bytestore

4.3.5. Description of Algorithms

Collaborative Filtering

Collaborative Filtering recommends products based on what similar customers have already bought. If two customers have bought a lot of the same things, there is a good chance they will like similar things going forward.

A table of which customers bought which products is built from the orders and order_items tables. For a given customer, the system compares them against every other customer using cosine similarity, finds the customers who look most similar, and recommends products those customers bought that the original customer has not.

Content-Based Filtering

Content-Based Filtering looks at the products themselves rather than other customers. Each product is described using attributes like category, brand, and price range, and a profile is built for each customer from the attributes of what they have bought or viewed. Products with similar attributes to that profile get recommended.

This works even for a brand-new customer who has bought just one product, since the system can immediately compare that one product's attributes, which collaborative filtering cannot do on its own.

Algorithm Steps

1. Collect customer data

The system collects the customer's purchase history for collaborative filtering, and the attributes of products they have bought or viewed for content-based filtering.

2. Retrieve product information

The system retrieves the available products and their attributes (category, brand, price range) from the database.

3. Calculate the Content-Based score

The system compares the customer's profile with the attributes of each product. A simplified content similarity score can be written as:

$$\text{ContentScore}(u, p) = \text{Similarity}(\mathbf{P}_u, \mathbf{F}_p)$$

Where:

$\mathbf{P}_u$ = the attribute profile built from products customer u has bought or viewed

$\mathbf{F}_p$ = the attribute vector of product p

A higher score means the product matches the customer's usual preferences more closely.

4. Identify similar customers

The system compares the target customer's purchase vector against every other customer's purchase vector to find customers with similar buying patterns.

5. Calculate the Collaborative score

The system calculates a recommendation score based on what similar customers have purchased:

$$\text{CollaborativeScore}(u,p)=\sum_{v\in N(u)} \text{Similarity}(u,v) \times R(v,p)$$

This helps surface products that were bought by customers with similar purchase history, even if the target customer has never looked at that product before.

6. Calculate the final recommendation score

The Content-Based and Collaborative scores are combined into one final score:

$$\text{FinalScore}(u, p)=\alpha \cdot \text{ContentScore}(u, p)+\beta \cdot \text{CollaborativeScore}(u, p)$$

Where:

α = weight given to Content-Based Filtering

β = weight given to Collaborative Filtering

$$\alpha + \beta = 1$$

For example, the system may use $\alpha = 0.5$ and $\beta = 0.5$, giving equal weight to both methods, depending on configuration.

7. Rank products

All candidate products are sorted by their final score:

RecommendedProducts = Sort(Products, by FinalScore descending)

8. Display recommendations

The highest-ranked products are shown to the customer as personalized recommendations.

Cosine Similarity

Both ContentScore and CollaborativeScore above depend on a Similarity(...) function, and that function is Cosine Similarity. It treats two things, whether customers or products, as vectors of numbers, and measures the angle between them.

$$\text{Cos}(\theta)=\frac{(A \cdot B)}{(|A| \times |B|)}$$

A and B are the two vectors being compared, $A \cdot B$ is their dot product, and $\|A\|$ and $\|B\|$ are their magnitudes. A result closer to 1 means high similarity, closer to 0 means little similarity.

Collaborative filtering example: Customer A has bought a laptop, a mouse, and a keyboard, while Customer B has bought a laptop, a mouse, and a headset. Represented as vectors, their dot product is 2, with magnitudes of about 1.73 each, giving a cosine similarity of roughly 0.667. Since Customer B also owns a headset, it becomes a reasonable recommendation for Customer A.

Content-based filtering example: A customer who bought a gaming laptop in a certain brand and price range will get a high similarity score, and a recommendation, for a similar gaming laptop from a different brand in the same category and price range, without needing any other customer's data.

Taken together, the two worked examples above show the same mathematical formula doing two different jobs: in Collaborative Filtering, the vectors represent what customers have bought, so a high score means two people shop alike; in Content-Based Filtering, the vectors represent what a product is, so a high score means two products are alike. ByteStore's FinalScore combines both readings into one ranked list, so a recommendation can be justified either by "people like you bought this" or by "this is similar to what you already like," whichever signal is stronger for that particular customer.

5. Gantt Chart

A Gantt chart is a bar chart that gives a visual view of tasks scheduled over time. It is used for planning projects of different sizes, and it helps show what work is scheduled for a given period, along with the start and end dates of the project overall. In this project, the Gantt chart below was built using Microsoft Excel.

Table 5.1:Scheduling Table for Bytestore

Phase	Start Date	End Date	Duration (days)
Requirement Analysis & Literature Review	Sep 14, 2026	Oct 11, 2026	28
System Design (ER, Use Case, Flow Chart)	Sep 28, 2026	Nov 1, 2026	35
Core Integration / DB Schema Extension	Oct 26, 2026	Nov 29, 2026	35
Collaborative & Content-Based Filtering Module	Nov 23, 2026	Jan 17, 2027	56
Testing & Integration	Jan 19, 2027	Feb 8, 2027	21
Documentation & Final Report Writing	Oct 12, 2026	Feb 14, 2027	126
Submission & Evaluation	Feb 16, 2027	Feb 16, 2027	1

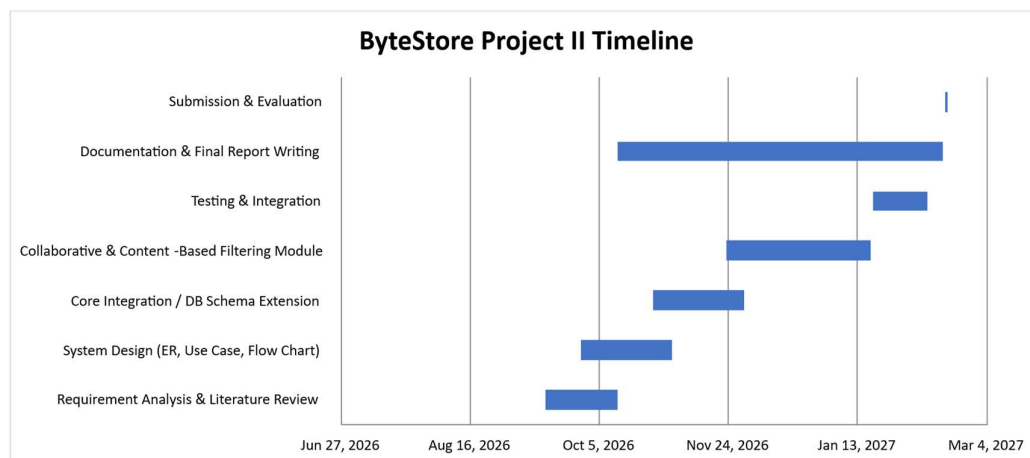


Figure 5.1: Gantt Chart for Bytestore

6. Expected outcome

By the end of this project, ByteStore should be able to give each customer a shopping experience that reacts to who they are, instead of showing the same catalog to everyone. Specifically, this project is expected to deliver:

- A working Collaborative Filtering module that looks at purchase history across customers and surfaces products bought by people with similar buying patterns.
- A working Content-Based Filtering module that looks at product attributes, so customers with little or no purchase history still get relevant suggestions instead of nothing.
- A combined recommendation list, shown to the customer, that draws on both methods rather than relying on just one.
- A small admin-facing view showing which products are being recommended and how often, giving the store some visibility into how the feature is performing.
- A working demonstration of cosine similarity applied to two different kinds of data, customer behavior and product attributes, using the same underlying formula.

Beyond the feature itself, this project is meant to show that a real, working system can be meaningfully improved after the fact, rather than needing to be rebuilt from the ground up every time something new needs to be added. It also gives a concrete, testable example of how similarity-based algorithms and object-oriented design come together in a normal, PHP-based web application, rather than staying purely theoretical.

7. References

- [1] Subbulakshmi, "PLING Adaptive Opportunity Checking of Web Service with Recommendation System," *International Journal of Pure and Applied Mathematics*, vol. 119, no. 16, pp. 247-253, 2018.
- [2] G. Linden, "Amazon.com Recommendations: Item-to-Item Collaborative Filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76-80, 2003.
- [3] Xuefeng, "A hybrid collaborative filtering method for multiple-interests and multiple-content recommendation in E-Commerce," *Expert Systems with Applications*, vol. 28, pp. 67-77, 2005.
- [4] DZhang, "A Personalized Collaborative Recommendation Algorithm Based on Hybrid Information," *4th International Conference on Mechatronics*, vol. Chemistry and Computer Engineering (ICMMCCE), 2015.
- [5] G. M and M. Challa, "Advancing E-Commerce Recommendations with Hybrid AI Models and Predictive Analytics," *International Journal of Intelligent Engineering and Systems*, vol. 18, p. 773, 2025.
- [6] Olist, "Brazilian E-Commerce Public Dataset," Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>.
- [7] M. Kechinov, "eCommerce events history in electronics store," Kaggle (REES46 Marketing Platform), 2020. [Online]. Available: <https://www.kaggle.com/datasets/mkechinov/ecommerce-events-history-in-electronics-store>.
- [8] H. Gharti Magar, "Building and Optimizing a Personalized Recommendation System for E-Commerce Using Machine Learning".
- [9] T. P. Alvin, "A Content-Based Recommender for E-Commerce Web Store," Towards Data Science, 2021. [Online]. Available: <https://towardsdatascience.com/a-content-based-recommender-for-e-commerce-web-store-7554b5b73eac>.